Operating System Questions

Q1. What is a process?

Answer:
A **process is a program that is currently running**. When we open a program, the operating system creates a process for it and gives it resources like memory and CPU time.

Example:
When you open Chrome, the OS creates one or more processes to run Chrome.

Q2. What is a thread?

Answer:
A **thread is the smallest unit of execution inside a process**. A process can have multiple threads, and these threads share the process's resources.

Example:
In a web browser, one thread can handle the user interface while another thread downloads a file.

Q3. Process vs Thread?

Answer:

Process	Thread
Independent program in execution	Execution unit inside a process
Has its own memory space	Shares memory with other threads
More expensive to create	Cheaper to create
Communication is relatively slower	Communication is faster

Example:
Think of a **restaurant as a process** and its **workers as threads**. Workers share the restaurant's kitchen and resources, while separate restaurants have their own resources.

Q4. What is a PCB?

Answer:
PCB stands for **Process Control Block**. It is a data structure maintained by the operating system that stores important information about a process.

It can contain:

- Process ID
- Process state
- Program counter
- CPU registers
- Scheduling information
- Memory information

Example:
Think of PCB as an **employee record** containing the employee's ID, current task, department, and other information needed to manage them.

Q5. What happens when a process is created?

Answer:
When a process is created, the OS generally:

1. Creates a PCB.
2. Assigns a process ID.
3. Allocates memory and other resources.
4. Loads the program into memory.
5. Places the process in the appropriate scheduling queue.
6. Eventually gives it CPU time.

Example:
When you open a calculator application, the OS creates a process, assigns it an ID, allocates memory, loads the application, and schedules it to run.

Q6. What is context switching?

Answer:
Context switching is the process of saving the state of one process or thread and loading the state of another, so the CPU can switch between them.

Example:
Suppose you're running Chrome and Spotify. The CPU works on Chrome for a while, saves its state, then switches to Spotify and loads Spotify's state.

Q7. Why is context switching expensive?

Answer:
Context switching takes time because the OS has to **save the current state and load another process or thread's state**. During this time, the CPU is doing switching work instead of useful application work.

Example:
Imagine a worker switching between two jobs. Every time they switch, they need to stop one task, remember where they were, prepare the other task, and continue. That switching takes time.

Q8. User-level vs Kernel-level threads?

Answer:
User-level threads are managed by a user-level library, while **kernel-level threads** are managed directly by the operating system.

User-level threads are generally faster to manage, but the OS has less visibility into them.

Kernel-level threads are managed by the OS, so the OS can schedule them individually.

Example:
User-level threads are like workers managed by a **team leader**, while kernel-level threads are like workers managed directly by the **company management**.

Q9. What is multithreading?

Answer:
Multithreading means running multiple threads within the same process. These threads can perform different tasks concurrently.

Example:
In a web application, one thread can handle user requests, another can process data, and another can perform logging.

Q10. What problems can occur with multiple threads?

Answer:
Multiple threads can cause problems such as:

- Race conditions
- Deadlocks
- Data inconsistency
- Synchronization problems

Example:
Two threads try to increase the same bank balance at exactly the same time. If they aren't synchronized properly, one update could overwrite the other.

Q11. What is CPU scheduling?

Answer:
CPU scheduling is the process of deciding which ready process or thread should get the CPU next. The operating system's scheduler makes this decision.

Example:
Imagine five people waiting to use one computer. The scheduler decides who gets to use the computer next.

Q12. FCFS vs SJF vs Round Robin?

Answer:

FCFS — First Come First Serve:
The process that arrives first gets the CPU first.

SJF — Shortest Job First:
The process with the shortest CPU burst is selected first.

Round Robin:
Each process gets a fixed amount of CPU time called a **time quantum**, and then the CPU moves to the next process.

Example:

Suppose three people are waiting:

A → needs 10 minutes
B → needs 2 minutes
C → needs 5 minutes

FCFS:

A → B → C

SJF:

B → C → A

Round Robin might give each person 2 minutes at a time:

A → B → C → A → C → A...

Q13. What is preemptive vs non-preemptive scheduling?

Answer:

Preemptive scheduling:
The OS can interrupt a running process and give the CPU to another process.

Non-preemptive scheduling:
Once a process gets the CPU, it keeps it until it finishes or voluntarily gives it up.

Example:
In a classroom, if the teacher can stop one student and ask another student to answer, that's like **preemptive scheduling**.

If the student must finish speaking before another student gets a turn, that's **non-preemptive scheduling**.

Q14. What is starvation?

Answer:
Starvation occurs when a process keeps waiting for CPU or another resource for a very long time because other processes keep getting priority.

Example:
Suppose high-priority tasks keep arriving. A low-priority task may continuously wait and never get CPU time.

Q15. What is aging?

Answer:
Aging is a technique used to prevent starvation by gradually increasing the priority of a process that has been waiting for a long time.

Example:
A low-priority process starts with priority 10. After waiting for a long time, its priority might be increased to 9, then 8, and so on until it gets CPU time.

Q16. How does Round Robin work?

Answer:
Round Robin gives each process a fixed amount of CPU time called a **time quantum**.

After the time quantum expires, the OS moves the process to the back of the queue and gives the CPU to the next process.

Example:

Suppose:

Processes: P1, P2, P3
Time quantum: 2 seconds

The CPU works like:

P1 → 2 sec
P2 → 2 sec
P3 → 2 sec
P1 → 2 sec
P2 → 2 sec
...

This gives processes a fair opportunity to run.

Q17. How do you calculate waiting time and turnaround time?

Answer:

Turnaround Time:

Turnaround Time = Completion Time - Arrival Time

Waiting Time:

Waiting Time = Turnaround Time - Burst Time

Example:

Suppose:

Arrival Time = 0
Burst Time = 5
Completion Time = 8

Then:

Turnaround Time = 8 - 0 = 8
Waiting Time = 8 - 5 = 3

So the process waited **3 units of time**.

Q18. What is a race condition?

Answer:
A **race condition occurs when multiple threads access shared data at the same time and the final result depends on the order in which they execute.**

Example:
Suppose two threads both increase:

counter = 10

Both read 10 and increase it to 11. The expected result is 12, but the actual result may become 11.

Q19. What is a critical section?

Answer:
A **critical section is the part of a program where shared resources are accessed or modified** and therefore needs proper synchronization.

Example:

counter++;

If multiple threads modify the same counter, this code can be a critical section.

Q20. What is a mutex?

Answer:
A **mutex is a synchronization mechanism that allows only one thread to access a critical section at a time.**

If one thread has the mutex, other threads have to wait.

Example:
Think of a **single bathroom with one key**. Whoever has the key can enter. Everyone else waits.

Q21. What is a semaphore?

Answer:
A **semaphore is a synchronization mechanism that uses a counter to control access to a shared resource.**

It can allow **multiple threads** to access a resource depending on its count.

Example:
Imagine a parking lot with **3 available spaces**. Three cars can enter, but the fourth car must wait until one space becomes free.

Q22. Mutex vs Semaphore?

Answer:

Mutex:

- Usually allows only one thread at a time.
- Used mainly for mutual exclusion.
- The thread that locks it generally unlocks it.

Semaphore:

- Can allow multiple threads.
- Uses a counter.
- Useful for controlling access to a limited number of resources.

Example:
Mutex = one bathroom key.

Semaphore = parking lot with 5 spaces.

Q23. What is a monitor?

Answer:
A **monitor is a high-level synchronization mechanism that combines shared data, operations on that data, and synchronization into one structure.**

It ensures that only one thread can execute the monitor's protected code at a time.

Example:
Think of a **bank counter** where customers access their account through controlled operations, and only one customer is served at the counter at a time.

Q24. What is a deadlock?

Answer:
A **deadlock occurs when two or more processes are waiting for resources held by each other, so none of them can continue.**

Example:

Process A → holds Resource 1 → waiting for Resource 2
Process B → holds Resource 2 → waiting for Resource 1

Both keep waiting forever.

Q25. What are the 4 conditions necessary for deadlock?

Answer:
The four conditions are:

1. **Mutual Exclusion** — A resource can be used by only one process at a time.
2. **Hold and Wait** — A process holds one resource while waiting for another.
3. **No Preemption** — A resource cannot be forcibly taken away.
4. **Circular Wait** — Processes form a circular chain where each waits for a resource held by the next.

Example:

A has Printer → wants Scanner
B has Scanner → wants Printer

A waits for B, and B waits for A.

Q26. Deadlock prevention vs avoidance vs detection?

Answer:

Prevention:
Design the system so that at least one of the four deadlock conditions cannot occur.

Avoidance:
Before allocating resources, check whether doing so could lead to an unsafe state.

Detection:
Allow deadlocks to happen and periodically check whether a deadlock exists.

Example:
Think about traffic:

- **Prevention:** Don't allow certain roads to create circular traffic.
- **Avoidance:** Before allowing a car onto a road, check whether it could cause a traffic jam.
- **Detection:** Let the traffic jam happen and then detect it.

Q27. What is the Banker's Algorithm?

Answer:
The **Banker's Algorithm is a deadlock-avoidance algorithm**. Before giving resources to a process, it checks whether the system will remain in a **safe state**.

If granting the request could make the system unsafe, the request is delayed.

Example:
A bank doesn't give a loan if doing so could leave it unable to satisfy the financial needs of its other customers. It checks whether it can safely grant the loan first.

Q28. What is virtual memory?

Answer:
Virtual memory is a memory-management technique that allows a process to use more memory than the available physical RAM by using part of the storage as an extension of memory.

Example:
Suppose your computer has **8 GB RAM**, but applications need more memory. The OS can use some SSD/storage as virtual memory.

It's much slower than RAM, but it allows the system to handle larger workloads.

Q29. What is paging?

Answer:
Paging divides physical memory into fixed-size blocks called frames and logical/virtual memory into fixed-size blocks called pages.

Pages can be loaded into any available frame.

Example:
Imagine a book divided into pages and a bookshelf divided into equally sized spaces. Any book page can be placed into any available space.

Q30. What is segmentation?

Answer:
Segmentation divides a program's memory into logical sections of different sizes, such as code, data, and stack.

Example:

Program
├── Code
├── Data
└── Stack

Each of these can be treated as a separate segment.

Q31. Paging vs Segmentation?

Answer:

Paging	Segmentation
Fixed-size blocks	Variable-size blocks
Divides memory into pages/frames	Divides program into logical segments
Mainly helps physical memory management	Represents logical program structure
Can cause internal fragmentation	Can cause external fragmentation

Example:
Paging is like dividing a book into **equal-sized pages**.

Segmentation is like dividing a book into **chapters of different lengths**.

Q32. What is a page fault?

Answer:
A **page fault occurs when a process tries to access a page that is not currently present in physical RAM.**

The OS then has to bring that page from storage into RAM.

Example:
Suppose an application needs Page 5, but Page 5 isn't currently in RAM. The OS fetches it from the disk/SSD. This event is called a **page fault**.

Q33. What is demand paging?

Answer:
Demand paging means pages are loaded into RAM only when they are actually needed, rather than loading the entire program into memory at once.

Example:
If an application has 100 pages but currently needs only 10, the OS doesn't have to load all 100 pages immediately.

Q34. What is thrashing?

Answer:
Thrashing occurs when the system spends most of its time moving pages between RAM and storage instead of executing actual processes.

It usually happens when there isn't enough physical memory for the active workload.

Example:
Imagine a student constantly taking books from a shelf, putting them back, and taking different books, but barely reading anything. That's similar to thrashing.

Q35. What is TLB?

Answer:
TLB stands for **Translation Lookaside Buffer**. It is a small, fast cache that stores recently used **virtual-to-physical address translations**.

It helps the CPU translate virtual addresses faster.

Example:
Imagine you frequently look up people's phone numbers. Instead of searching your entire contact book every time, you keep frequently used numbers on a small sticky note. That sticky note is like the TLB.

Q36. What is page replacement?

Answer:
Page replacement is the process of selecting which page in RAM should be removed when a new page needs to be loaded and there is no free frame.

Example:
Suppose RAM can hold only 3 pages:

[A] [B] [C]

Now Page D is needed. The OS must decide which of A, B, or C should be removed.

Q37. FIFO vs LRU vs Optimal page replacement?

Answer:

FIFO — First In, First Out:
Remove the page that entered memory first.

LRU — Least Recently Used:
Remove the page that hasn't been used for the longest time.

Optimal:
Remove the page that will not be used for the longest time in the future.

Example:

Suppose RAM contains:

A B C

If A was loaded first:

FIFO:
Remove A.

If B hasn't been used for the longest time:

LRU:
Remove B.

If we somehow know that C won't be used for a very long time:

Optimal:
Remove C.

Important interview point:
Optimal page replacement gives the **best possible result**, but it requires knowledge of future memory accesses, so it isn't practically implementable in a real system. It is mainly used as a benchmark to compare other algorithms.